



Sistemi Distribuiti

Laurea in Informatica

Marco Savi

marco.savi@unimib.it



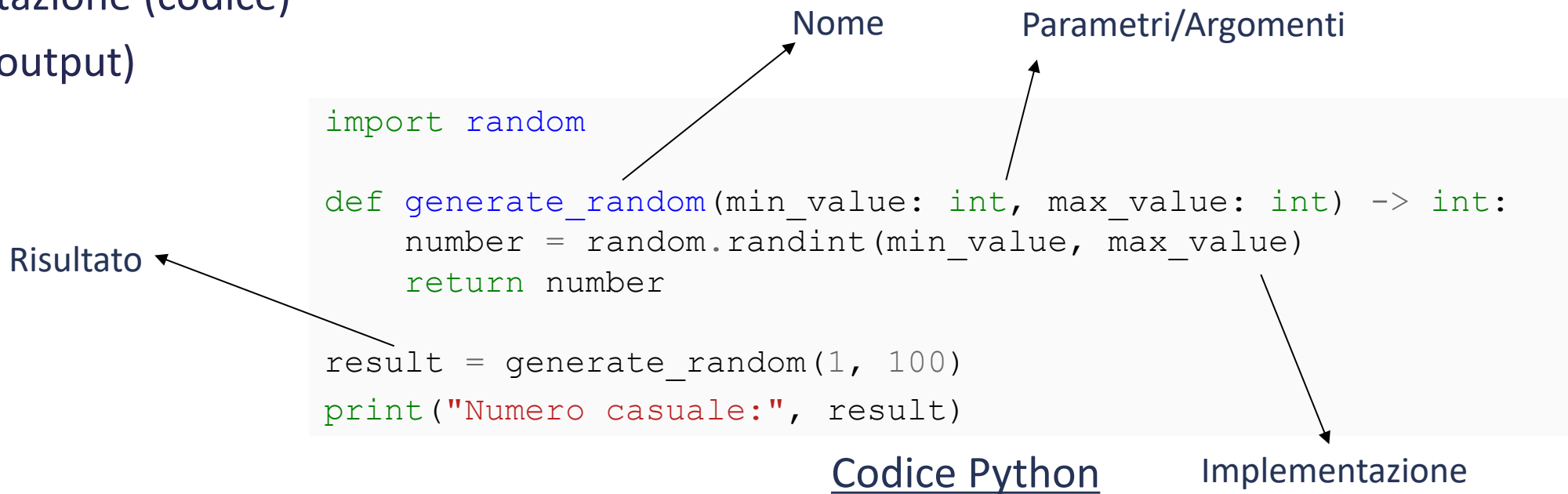
Remote Procedure Call (RPC)

Chiamata di Procedura Remota

- Approccio visto finora nella **comunicazione**: scambio di dati **esplicito** tra processi remoti
 - La comunicazione non è in alcun modo “nascosta”
- Alternativa: permettere ai programmi in esecuzione di chiamare procedure localizzate su macchine remote
 - **Remote Procedure Call (RPC)** → Lo scambio di dati con RPC è trasparente al programmatore
- **Vantaggi:**
 - La semantica è ben nota (quella delle chiamate di procedura «convenzionali»)
 - Possono facilmente sposare il modello client-server
- **Svantaggi:**
 - L'esecuzione delle procedure su macchine diverse implica che sono eseguite in spazi di indirizzamento differenti
 - I parametri e i risultati devono essere passati tra le macchine (non banale, soprattutto se le macchine non sono identiche)
 - Recuperare dal fallimento di una delle due macchine non è semplice

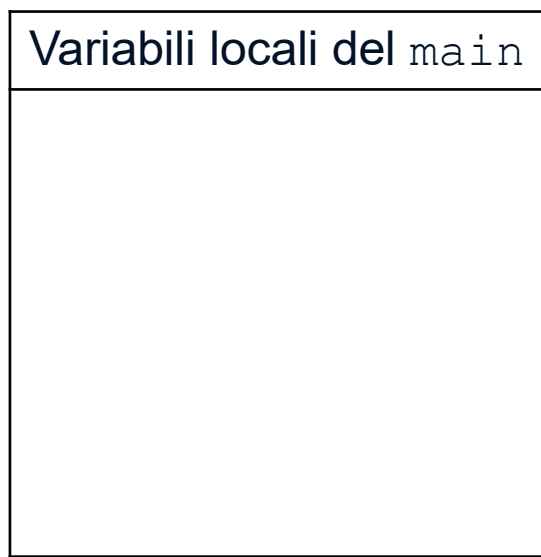
Chiamata di Procedura Convenzionale

- I programmi invocano **procedure** (o **funzioni**, o **metodi** di una classe)
- Una **procedura** è definita da:
 - Nome
 - Parametri/Argomenti (input)
 - Implementazione (codice)
 - Risultato (output)



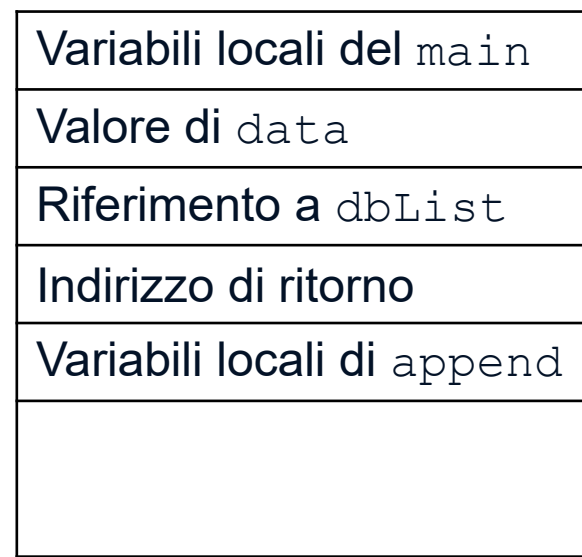
Chiamata di Procedura Convenzionale – Passaggio parametri

- **Esempio:** effettuare l'operazione `newList=append(data, dbList)`
- Assunzione:
 - `data` è rappresentato direttamente dal suo **valore**
 - `dbList` è rappresentato da un **riferimento** (es. puntatore) a un oggetto del tipo "lista"
- Risultato sullo stack di una chiamata ad `append`:



← Stack pointer

Prima della chiamata

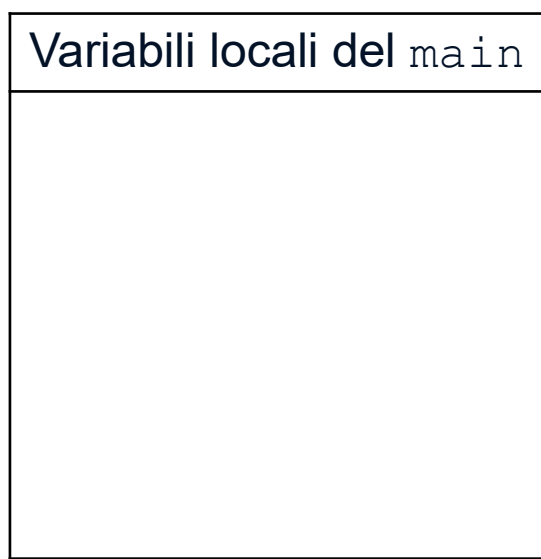


← Stack pointer

Dopo la chiamata

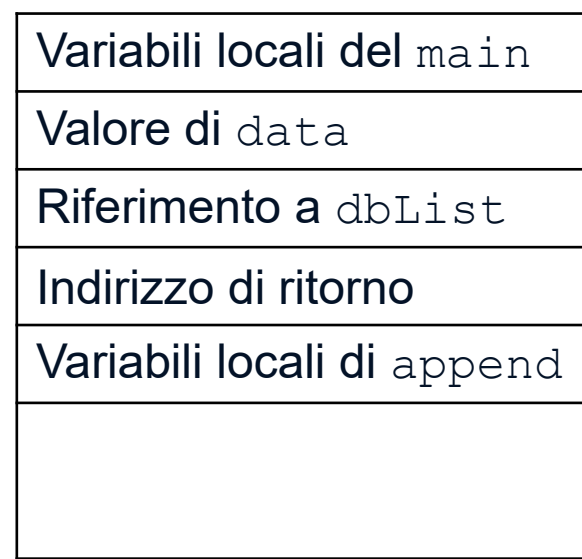
Chiamata di Procedura Convenzionale – Passaggio parametri

- **Esempio:** effettuare l'operazione `newList=append(data, dbList)`
- Differenti policy nel passaggio parametri per `data` e `dbList`
 - `Data`: policy **call-by-value** (chiamata per valore)
 - `dbList`: policy **call-by-reference** (chiamata per riferimento)



← Stack pointer

Prima della chiamata

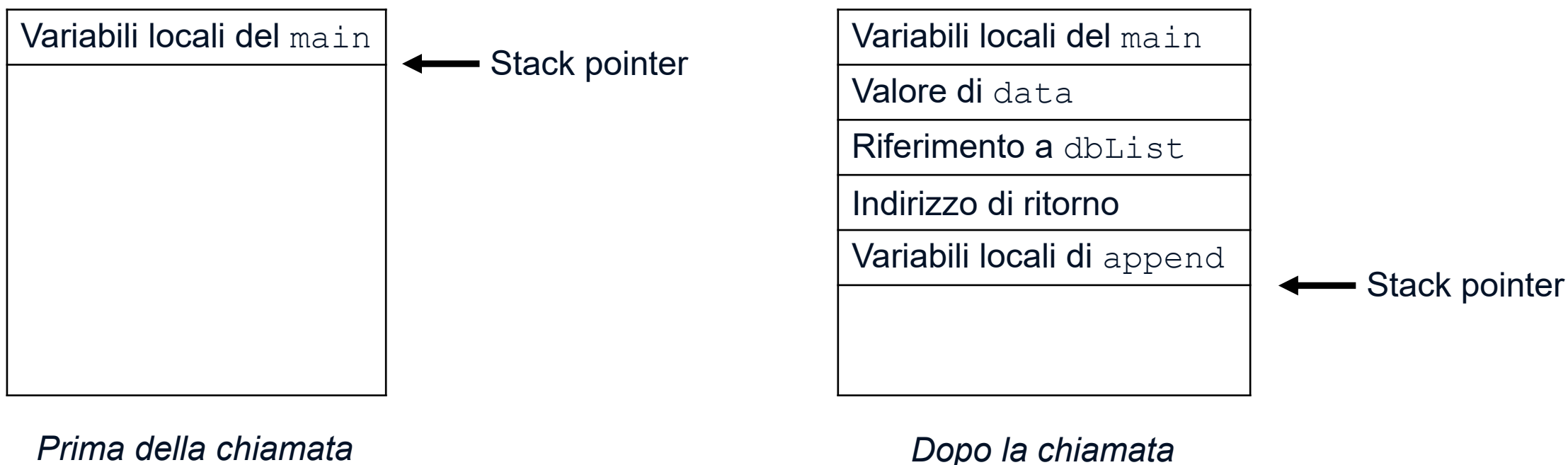


← Stack pointer

Dopo la chiamata

Chiamata di Procedura Convenzionale – Passaggio parametri

- **Esempio:** effettuare l'operazione `newList=append(data, dbList)`
- Effetto di modifiche a `data` e `dbList` da parte di `append`:
 - `data` è una variabile locale → Il valore originale lato chiamante (`main`) non è affetto
 - `dbList` è un riferimento a un oggetto (include un indirizzo di memoria) → Quando il valore `data` è aggiunto alla lista, l'oggetto viene modificato!



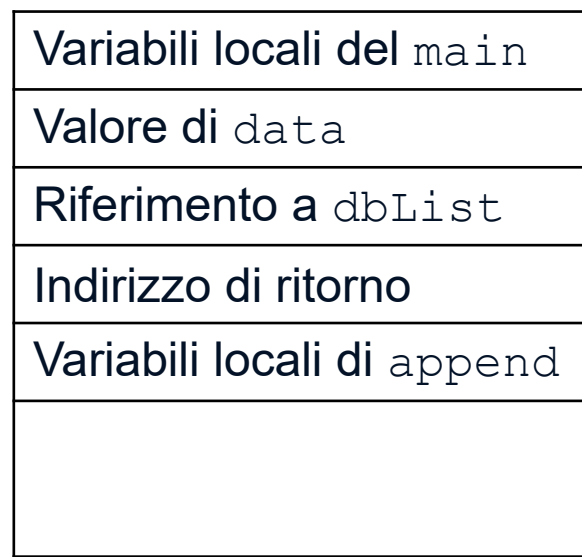
Chiamata di Procedura Convenzionale vs. Remota

- Nel caso di RPC un approccio di tipo call-by-reference è problematico!
 - Gli indirizzi hanno valenza locale, quindi un riferimento ha senso solo sulla stessa macchina!
- Meccanismo alternativo: **call-by-copy/restore** (chiamata per copia/ripristino)
 1. Il chiamante copia la variabile nello stack (come nel call-by-value)
 2. Terminata la chiamata viene sovrascritto il valore originale della variabile con quello di tale copia
- E' una valida alternativa a call-by-reference nel caso di RPC



← Stack pointer

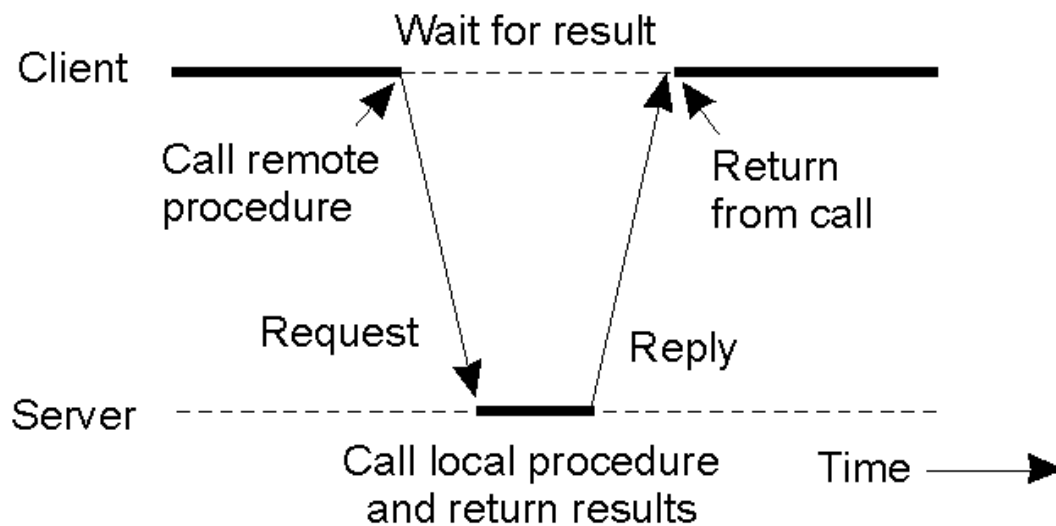
Prima della chiamata



← Stack pointer

Dopo la chiamata

Il modello RPC



- Il modello RPC “maschera” il modello client-server
- In che modo RPC garantisce trasparenza?
 - Il chiamante (`main`) lato client chiama la procedura `append` nel solito modo
 - Viene scatenato un meccanismo per cui i parametri di `append` (`data` e `dbList`) vengono inseriti in una richiesta da inviare al chiamato (la procedura remota `append`) lato server
 - Il chiamante blocca l'esecuzione finché non ha ottenuto una risposta (il risultato dell'operazione)
 - Livello di applicazione: esecuzione sincrona (esecuzione sospesa fino alla ricezione del risultato)
 - Livello di comunicazione: comunicazione transiente sincrona basata sulla risposta (si attende la risposta)

Da Chiamata di Procedura a Chiamata di Procedura Remota

- Una Remote Procedure Call deve quindi essere gestita da un **middleware** che si preoccupa di trasformare le chiamate locali in chiamate remote
- Vari interrogativi a cui bisogna dare risposta

Lato client

- Programma: chi chiamare?
- Middleware: come interagire con il programma?
- Middleware: dove si trova la macchina server?
- Middleware: come trattare i parametri?
- Middleware: come passare il risultato al programma?

Lato server

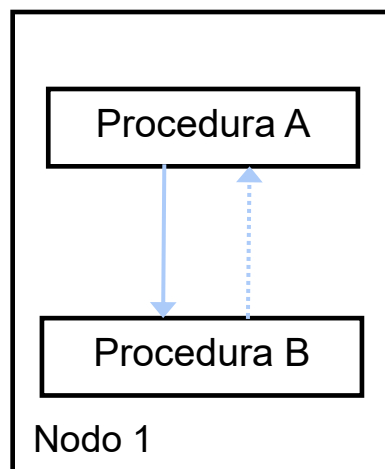
- Middleware: quale procedura invocare?
- Middleware: come passare i parametri?
- Procedura remota: a chi restituire il risultato?
- Middleware: come trattare il risultato?

Architettura RPC

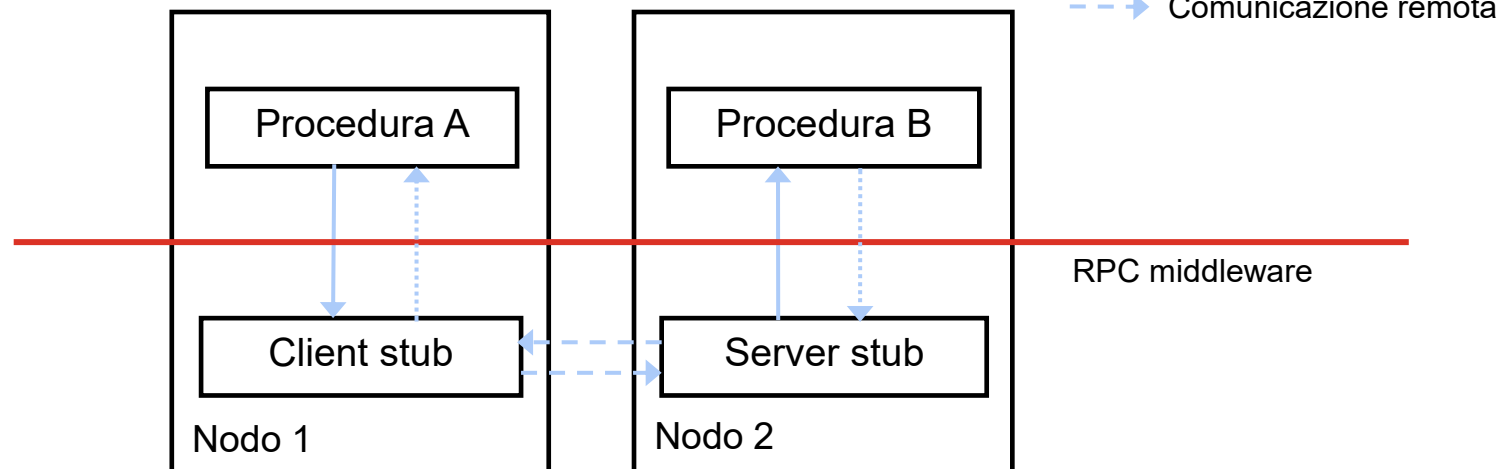
- Si utilizzano degli **stub** (o **proxy**, o **skeleton**) per
 - Simulare comportamenti locali per chiamante e chiamato
 - Realizzare la comunicazione tra processi remoti

Nell'esempio:

- Procedura A = `main`
- Procedura B = `append`



Chiamata di procedura



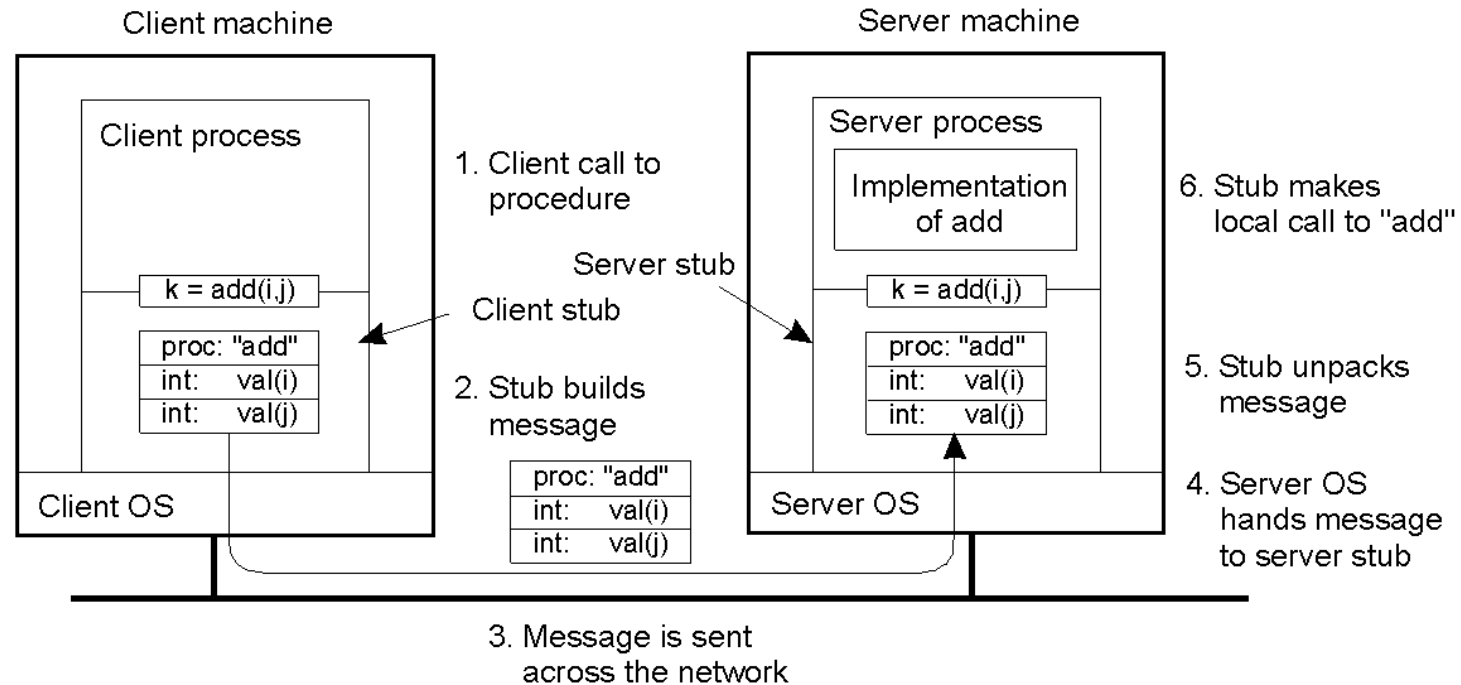
Chiamata di procedura remota (RPC)

Legenda:

- > Chiamata locale
- ...> Risultato
- -> Comunicazione remota

- **Client stub** (o **proxy**): offerto al chiamante (`main`) al momento della chiamata di `append`
 - Crea e invia al server la richiesta, includendo i parametri di `append` che gli sono stati passati
 - Attende finché non riceve risposta, quindi ritorna il risultato al chiamante
- **Server stub** (o **skeleton**): trasforma la richiesta ricevuta in una chiamata di procedura locale ad `append`, quindi crea e invia al client la risposta includendo il risultato

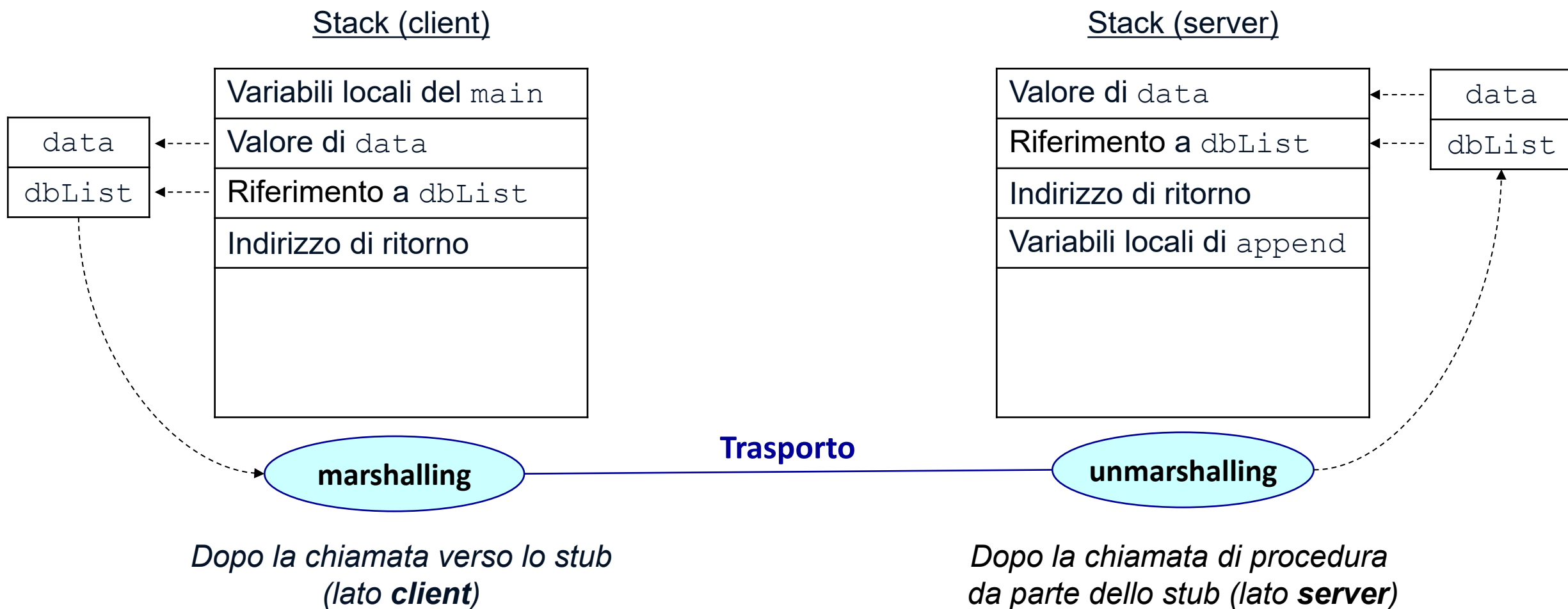
Passaggio dei parametri in RPC – Dettagli



- Altro esempio: operazione `k=add(i,j)`
- L'operazione di trasformazione delle strutture dati da inviare in un formato adatto alla trasmissione prende il nome **serializzazione** (effettuato lato client)
 - L'operazione inversa si chiama **deserializzazione** (effettuata lato server)
- Il processo di impacchettamento dei dati per il loro invio prende il nome di **marshalling** (effettuato lato client)
 - L'operazione inversa prende il nome di **unmarshalling** (effettuato lato server)
- **Nota:** il formato e la codifica dei dati devono essere condivisi tra client e server stub!
 - Interface Definition Language (IDL, es. ProtoBuf)

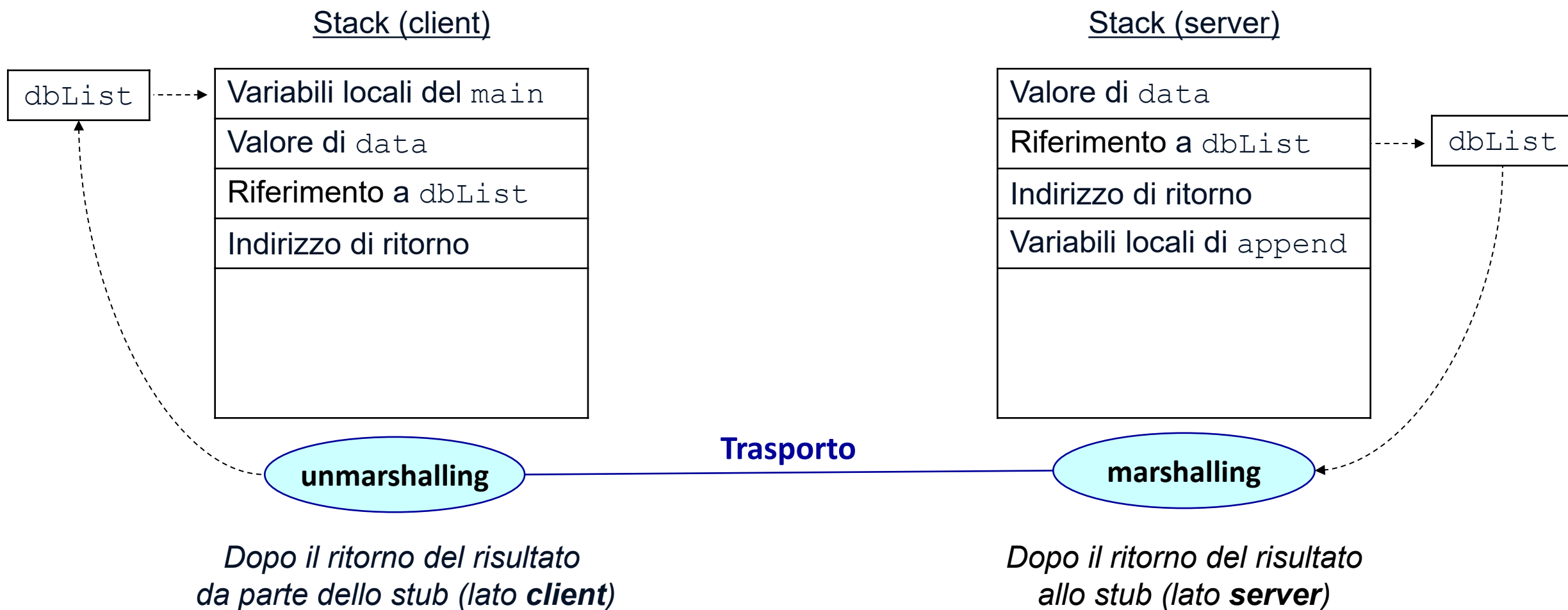
Passaggio dei parametri in RPC – Esempio

- Rimaniamo sull'esempio: `newList=append(data, dbList)`
- Come avviene il passaggio dei parametri?



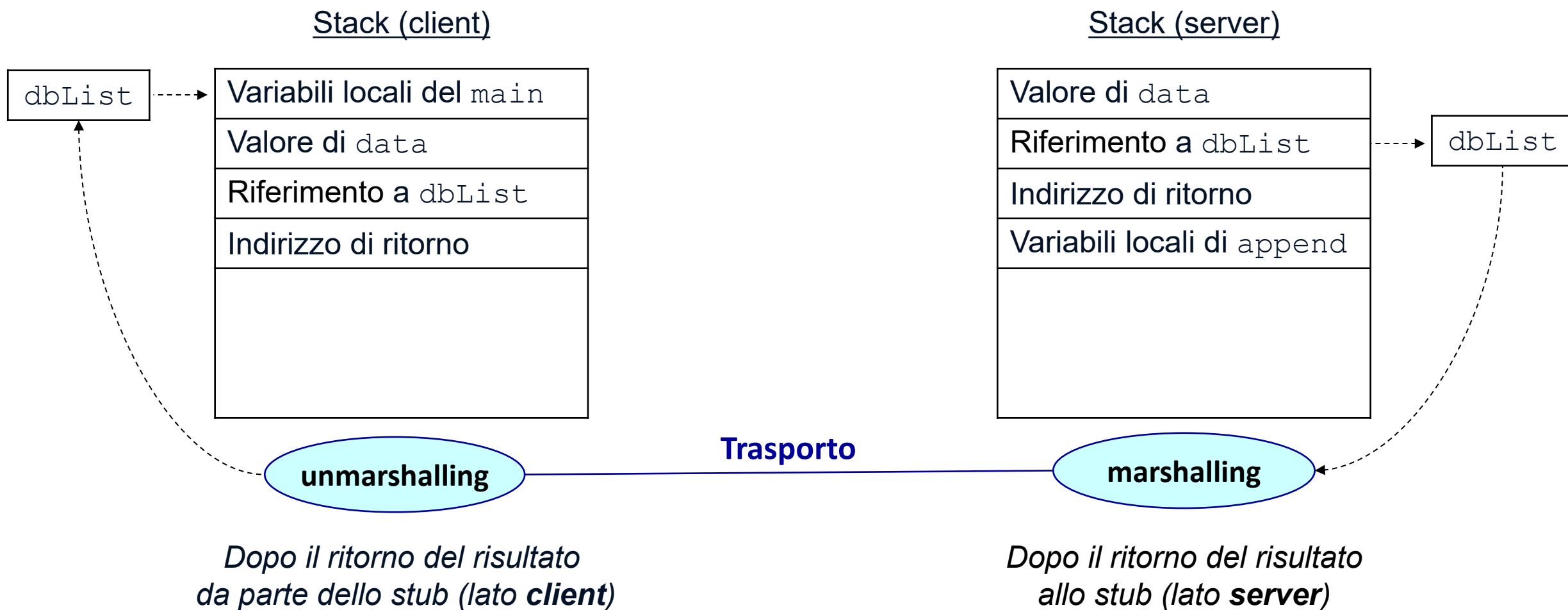
Passaggio dei parametri in RPC – Esempio

- Una volta che `dbList` è modificato dal server, viene impacchettato nella risposta
- Il client sovrascrive la sua copia locale di `dbList` → Approccio **call-by-copy/restore**

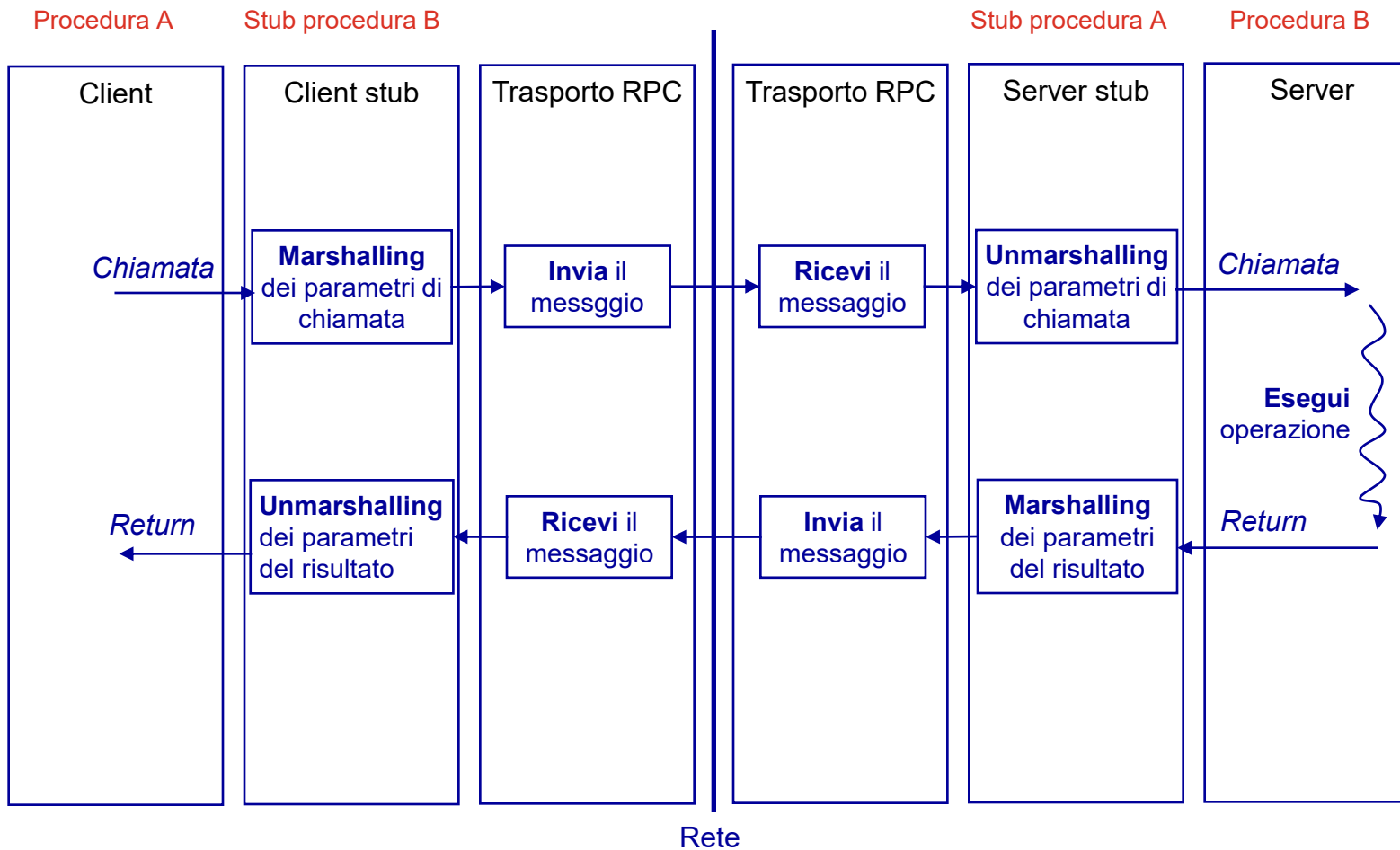


Passaggio dei parametri in RPC– Esempio

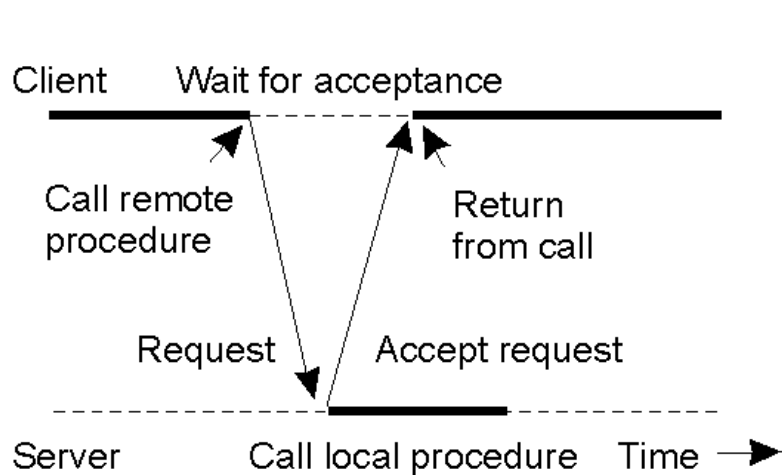
- **Nota:** l'effetto sul client è analogo a una call-by-reference!
 - Ma la modifica al valore referenziato è avvenuta dopo una chiamata a procedura remota



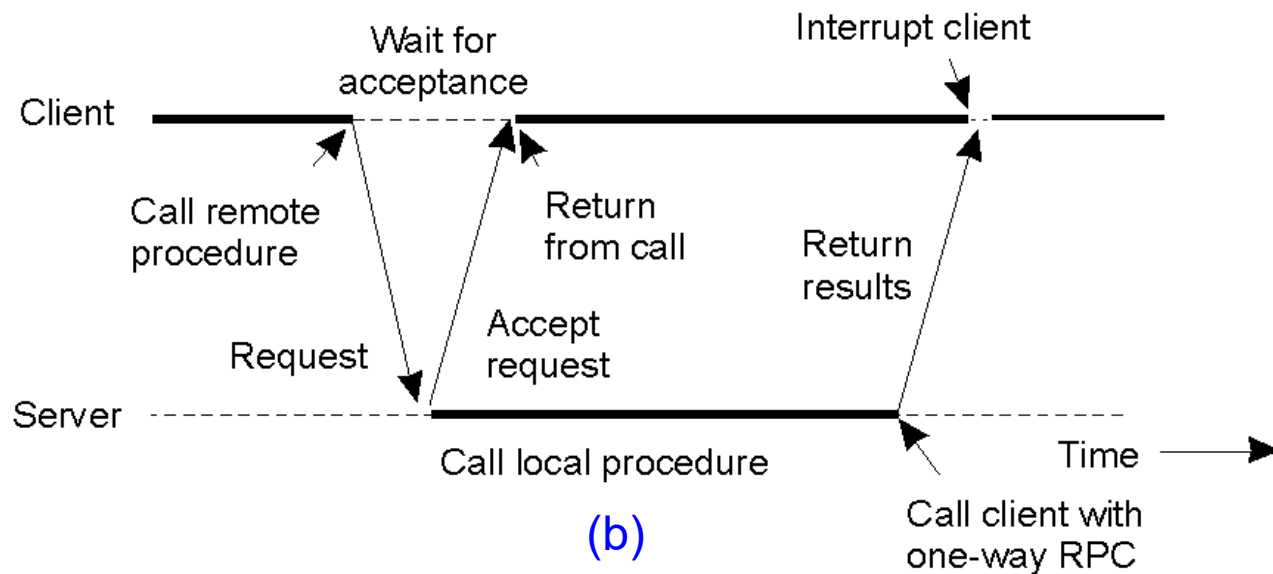
Esecuzione di una RPC – Recap



Chiamate RPC asincrone



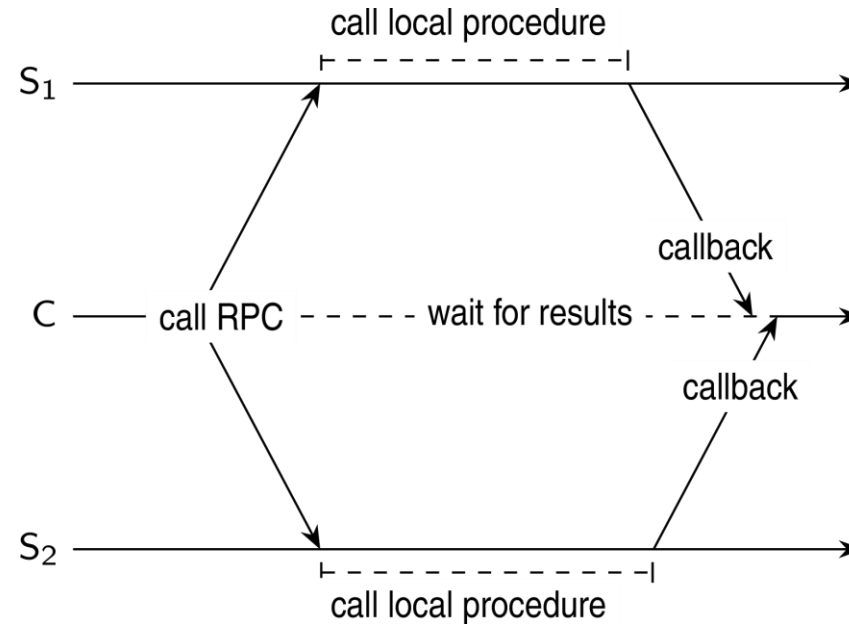
(a)



(b)

- **Caratteristica:** il client non deve attendere che il server completi l'esecuzione della chiamata per continuare l'esecuzione
 - Livello di applicazione: esecuzione asincrona (l'esecuzione continua senza attendere il risultato)
 - Livello di comunicazione: comunicazione transiente sincrona basata sulla ricezione (o sulla consegna)
- Due modelli:
 - a) **RPC asincrona senza risposta:** utile quando la procedura chiamata non deve restituire risultati
 - b) **RPC sincrona differita:** il client stub invoca una funzione di callback non appena riceve la risposta dal server stub (per mezzo di una RPC unidirezionale)
 - **RPC unidirezionale (one-way RPC):** non richiede un ack

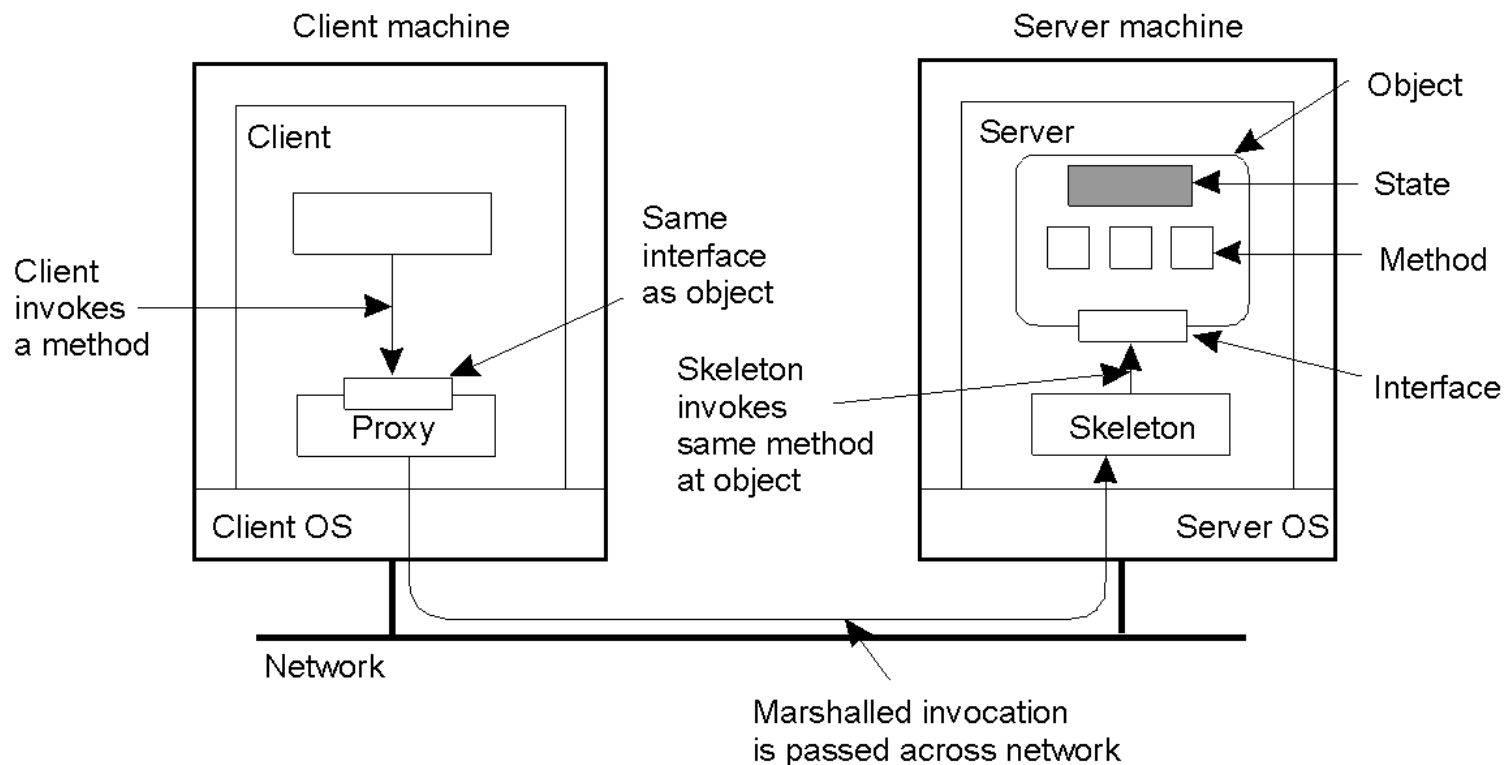
Chiamate RPC multicast



- Esecuzione di più RPC contemporaneamente
 - Invio di una richiesta RPC a un **gruppo di server**
- Ogni server elabora la richiesta ricevuta in modo indipendente (e in parallelo)
- Quando il risultato è restituito al client, viene invocata una callback dal client stub
- Due tipici utilizzi:
 - Garantire maggior tolleranza ai guasti (fault tolerance)
 - Distribuzione del carico (operazioni identiche ma su differenti parti dell'input o su basi di dati diverse)

Un passo oltre: chiamate remote in sistemi basati su oggetti

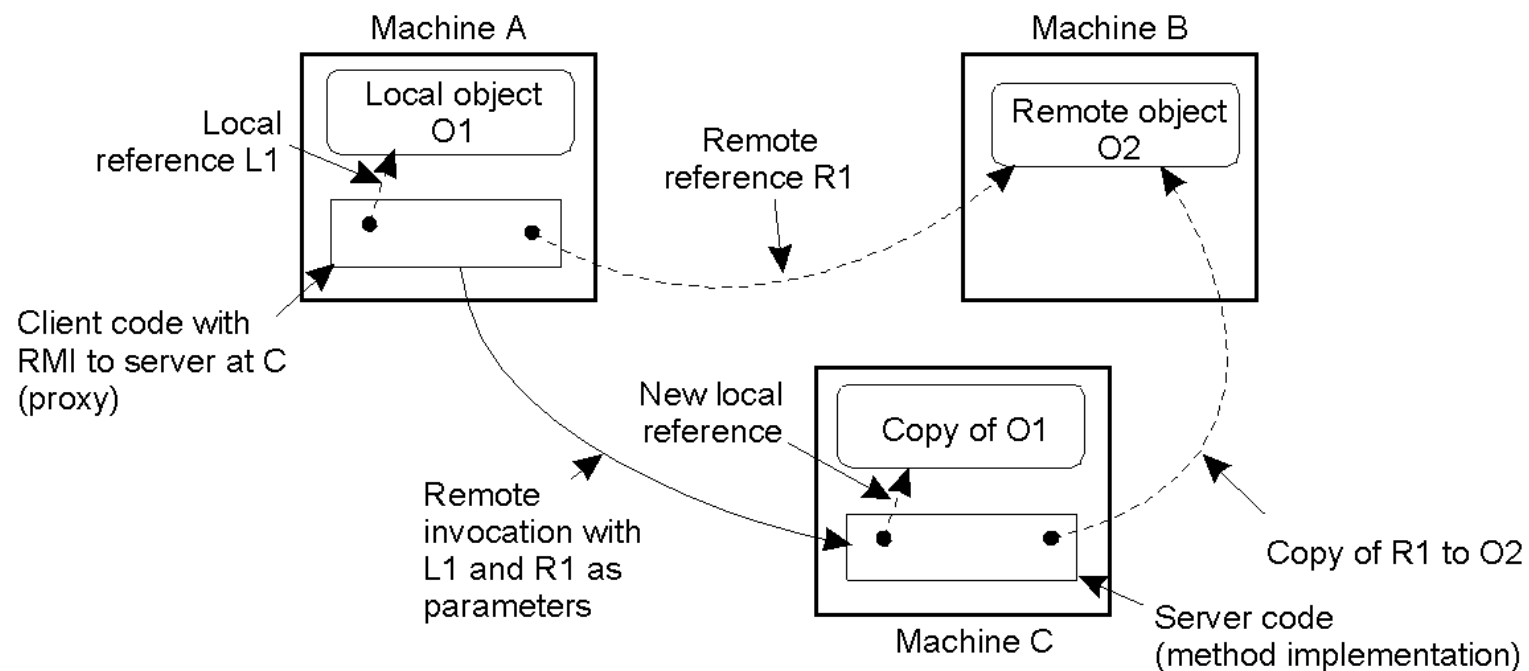
- **Obiettivo:** invocare un metodo di un oggetto remoto
- **Quali parametri** vengono passati e **come** in questo caso?
 - Cosa fare se un parametro da passare è a sua volta un oggetto?



Chiamate remote in sistemi basati su oggetti

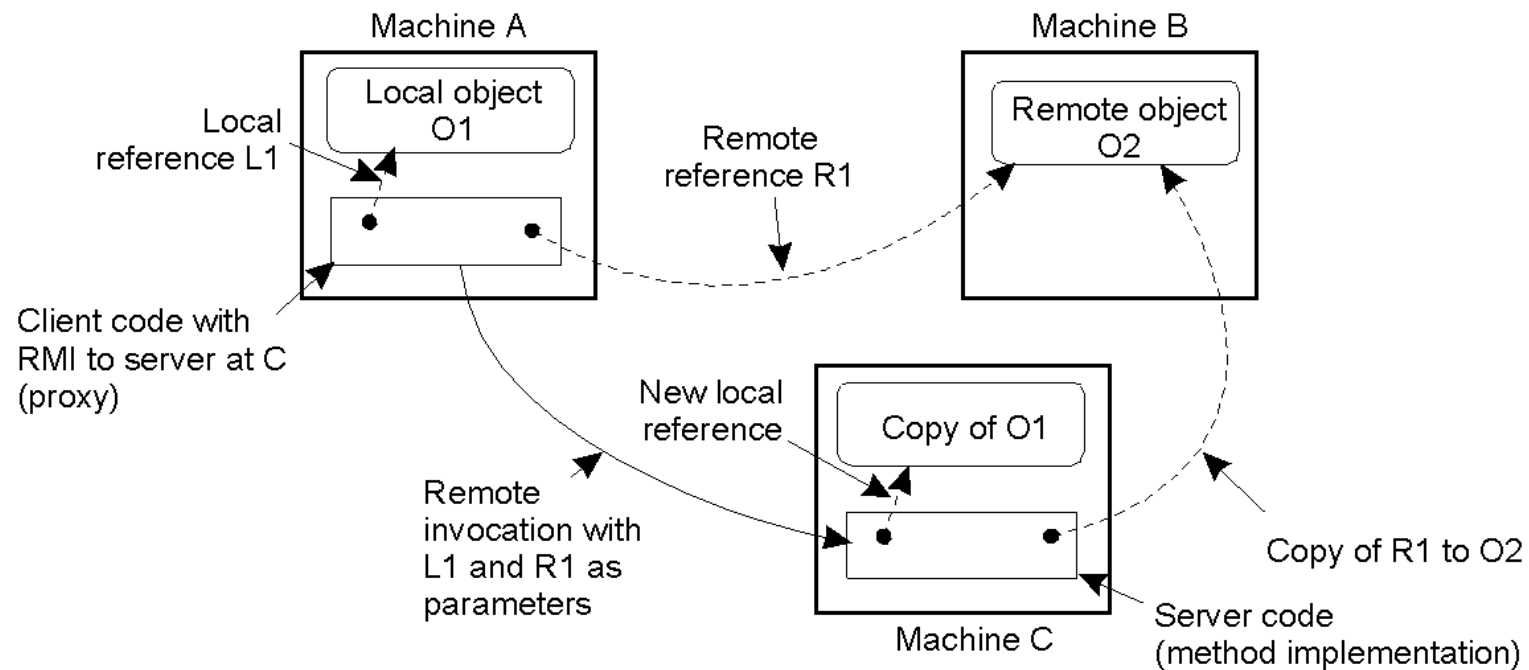
- Se si vuole fare in modo che tutti gli oggetti del sistema siano accessibili da machine remote è possibile utilizzare **referimenti remoti**
 - Esempio: {**Indirizzo IP** della macchina, **porta** associata al processo, **ID** dell'oggetto}
 - Possono essere **passati per valore** come parametri al momento di una chiamata di un metodo di un ulteriore oggetto remoto
- **Problema:** trattare tutti gli oggetti come oggetti remoti (anche quelli co-locati) nel passaggio parametri è oneroso e inefficiente
- **Passaggio di riferimenti a oggetti come parametri:** l'approccio tipico è di trattare in modo differente i riferimenti a oggetti remoti da quelli a oggetti locali
 - Oggetto remoto: riferimento passato per valore (oggetto passato per riferimento)
 - Oggetto locale: oggetto copiato e passato durante l'invocazione (oggetto passato per valore)

Chiamate remote in sistemi basati su oggetti



- Programma client: eseguito su A – Programma server: eseguito su C
 - L1: riferimento locale all'oggetto O1 (su macchina A)
 - R1: riferimento remoto all'oggetto O2 (su macchina B)
- Quando viene chiamato il metodo sul server (macchina C) con parametri L1 e R1:
 - Una copia di O1 è passata al server
 - Una copia del riferimento R1 a O2 è passata al server

Chiamate remote in sistemi basati su oggetti



- Questo meccanismo è alla base delle chiamate a metodi di oggetti remoti (**Remote Method Invocation – RMI**)
 - In Java è implementato in modo altamente trasparente
 - La differenza tra oggetti locali e remoti è visibile solo perché sono di tipo diverso

Alcune implementazioni di RPC

■ Generali:

- JSON-RPC
- XML-RPC e SOAP
- Apache Thrift
- gRPC

■ Per linguaggio:

- Java Remote Method Invocation (RMI)
- RPyC (per Python)
- Go RPC

■ Per applicazione:

- Kafka RPC (Messaggistica)
- Unreal Engine RPC (Videogiochi)
- Ray RPC (High Performance Computing e Machine Learning)

Implementazioni di RPC generali

Caratteristica	XML-RPC / SOAP	JSON-RPC	gRPC	Apache Thrift
Formato dati / Serializzazione	XML (Testuale)	JSON (Testuale)	ProtoBuf (Binario)	Binario, JSON, Compresso
Leggibilità	Alta (leggibile da un umano)	Molto Alta (semplice)	Bassa (codificato in bit)	Bassa o Molto Alta (dipende da formato)
Peso dei messaggi	Molto elevato (verboso)	Medio	Minimo (compattato)	Minimo o Medio (dipende da formato)
Velocità	Lenta (parsing XML pesante)	Media	Massima	Massima o Media
Trasporto	HTTP	TCP, HTTP	HTTP/2	TCP, HTTP



Introduzione a gRPC

Caratteristiche di gRPC

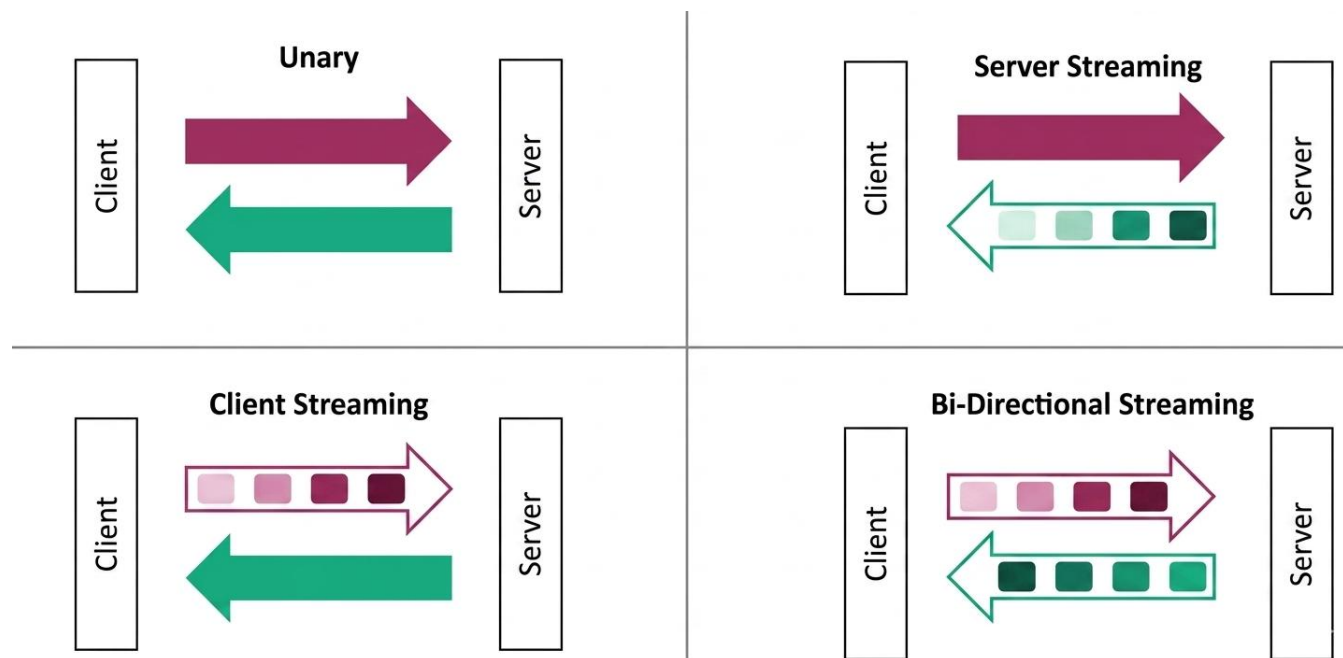


- E' un framework per RPC open source (<https://grpc.io/>)
 - Sviluppato da Google (2015)
 - Release attuale: 1.80.0 (glimmering, 26 marzo 2026)
- Garantisce **prestazioni elevate**
 - Utilizza Protocol Buffers (Protobuf) come IDL
 - Serializzazione binaria (dati inviati come bit compressi)
 - E' necessario scrivere un file `.proto` in cui è specificato esplicitamente il tipo di ogni dato da serializzare, eliminando ambiguità di interpretazione tra client e server
- Supporta nativamente **HTTP/2**
 - Possibile inviare più richieste e risposte contemporaneamente sulla stessa connessione TCP (multiplexing)
 - Compressione degli header HTTP (efficienza di banda)

Funzionalità di gRPC

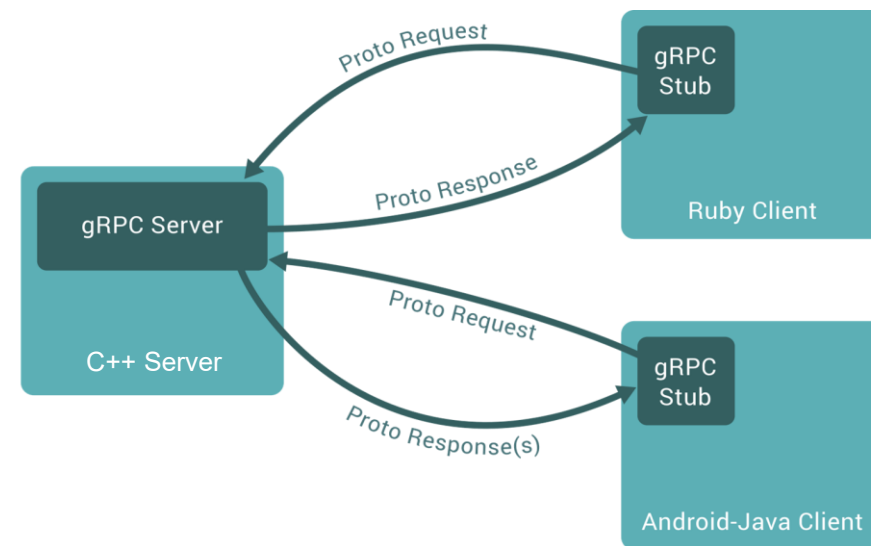
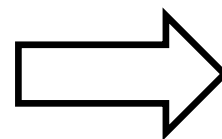
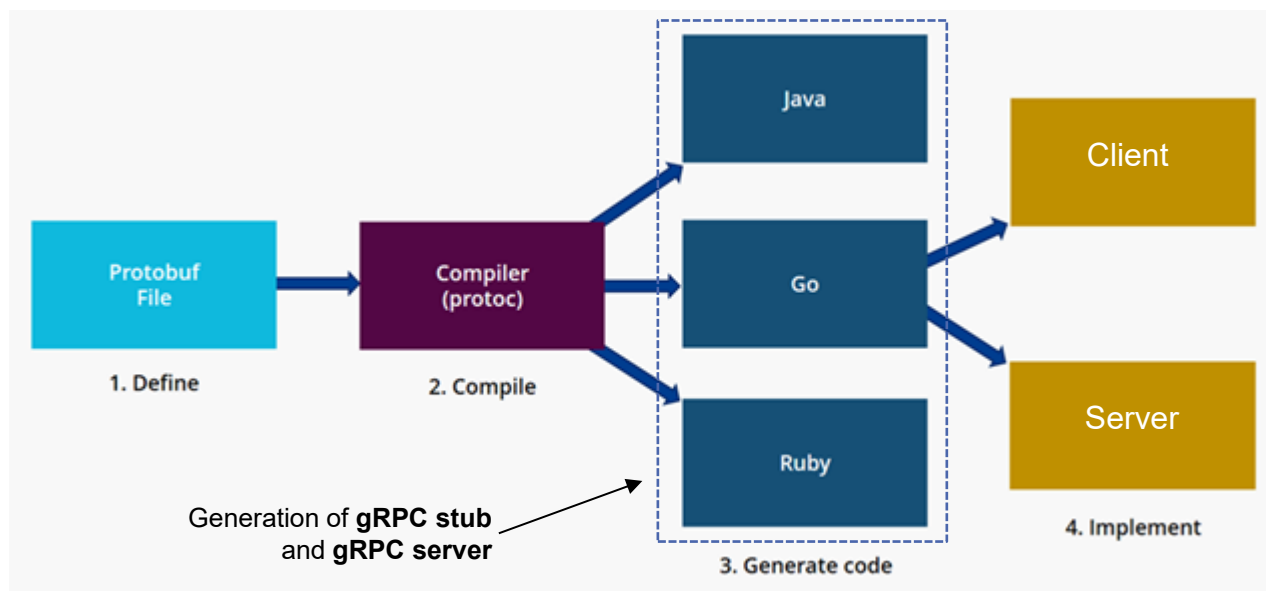
- Permette quattro **modelli di comunicazione**:

1. **Unary RPC**: classico schema richiesta-risposta (visto fino ad ora)
2. **Server Streaming**: il client invia una richiesta e il server risponde con un flusso continuo di messaggi (es. feed di borsa)
3. **Client Streaming**: il client invia un flusso di messaggi e il server risponde una volta sola alla fine (es. upload di file)
4. **Bidirectional Streaming**: entrambi inviano flussi di messaggi simultaneamente (es. chat)



Funzionalità di gRPC

- Con gRPC non bisogna scrivere manualmente il codice per marshalling e unmarshalling!
- Procedimento:
 - Si scrive un **file** `.proto`: definisce i **messaggi** (strutture dati) e i **servizi** (funzioni chiamabili remotamente)
 - Il codice viene compilato per mezzo del **compilatore protoc**
 - Viene generato automaticamente lo stub sia lato client (**gRPC stub**) che server (**gRPC server** o skeleton)
 - Vari linguaggi supportati (Java, Go, C++, Ruby, Python, ...)



Esempio di file .proto

- **Esempio:** richiesta verso un sistema IoT per il monitoraggio dei parametri dell'aria
 - Si vuole poter o chiedere lo stato attuale (Unary RPC) oppure ricevere aggiornamenti costanti (Streaming RPC)

```
1. // Specifica la versione della sintassi (proto3 è lo standard attuale)
2. syntax = "proto3";

3. // Definizione dei messaggi: o i dati specificati dall'utente (SensorRequest) o creati dal sensore (EnvironmentData)
4. // I numeri (1, 2, 3) sono i «Tag», ovvero identificatori binari delle variabili usati in fase di comunicazione
5. message SensorRequest {
6.     int32 sensor_id = 1; // ID del sensore specifico
7. }
8. message EnvironmentData {
9.     float temperature = 1; // Temperatura
10.    float humidity = 2; // Umidità in percentuale
11.    string status = 3; // Es. "OTTIMALE", "ALLERTA"
12. }

13. // Definizione del servizio: le procedure (funzioni) disponibili
14. service SystemMonitor {
15.    // 1. Unary RPC: Il client chiede i dati e riceve una sola risposta
16.    rpc GetCurrentStatus(SensorRequest) returns (EnvironmentData);
17.    // 2. Server Streaming: Il client chiede di monitorare e il server invia un flusso continuo di dati (es. ogni sec.)
18.    rpc WatchEnvironment(SensorRequest) returns (stream EnvironmentData);
19. }
```

Altre funzionalità integrate di gRPC

- **Timeout:** la chiamata viene annullata automaticamente dopo un certo tempo
- **Cancellazione della chiamata:** viene inviata una notifica al server, il quale può smettere di lavorare
- **Autenticazione:** supporto nativo (es. SSL/TLS)



Java RMI

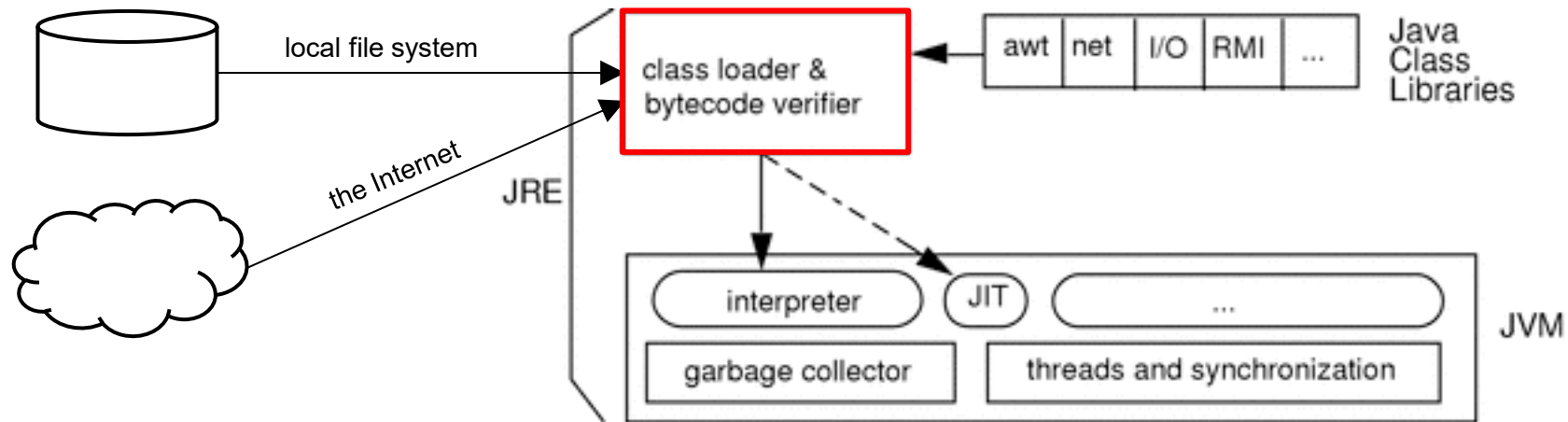
Introduzione a Java RMI

- E' un middleware che consente a oggetti su una Java Virtual Machine (JVM) di accedere a o invocare un oggetto in esecuzione su un'altra JVM
 - E' un'implementazione di RPC in Java
 - Abilita una comunicazione remota tra programmi Java trasparente al programmatore
- Viene fornito dal pacchetto `java.rmi`
- **Vantaggi** (rispetto a implementazioni RPC generali):
 - Supporto nativo a un design orientato agli oggetti
 - Integrazione totale e trasparente con il linguaggio Java
- **Svantaggi** (rispetto a implementazioni RPC generali):
 - Legato indissolubilmente a Java
 - Overhead maggiore e prestazioni minori
 - Maggior complessità a livello di rete e problemi di sicurezza

Architettura di un'applicazione RMI

- Vengono scritti due programmi: un **programma client** e un **programma server**
 - Nel programma server viene creato un **oggetto remoto**
 - Un **referimento** a tale oggetto viene messo a disposizione del client
 - Il programma client richiede **l'accesso all'oggetto remoto** e cerca di **invocare i suoi metodi**
- Problematiche fondamentali:
 - Come passare i parametri a un oggetto?
 - Come passare il riferimento remoto relativo a un oggetto?
 - Nel caso di riferimento locale, come passare lo stato dell'oggetto?
 - Nel caso di riferimento locale, come ricostruire la struttura dell'oggetto lato server?

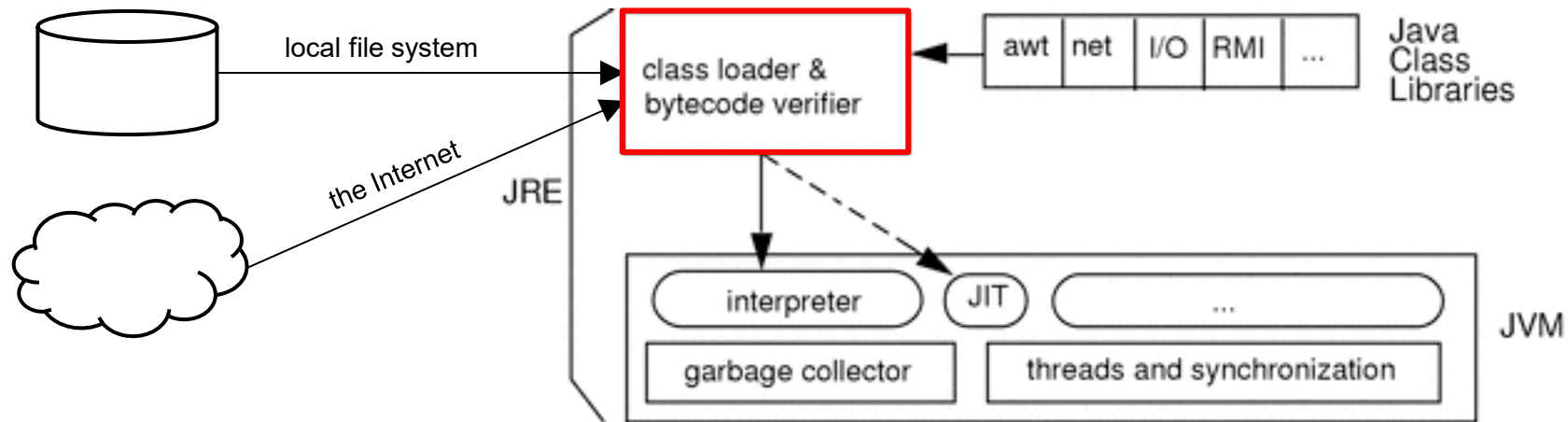
Ricostruzione struttura dell'oggetto – Java Runtime Environment



Fonte: <https://docs.oracle.com/cd/E19455-01/806-3461/6jck06gqb/index.html>

- In Java il problema della ricostruzione della struttura dell'oggetto è risolvibile nativamente grazie al Java Runtime Environment
- Il **class loader** ha il compito di caricare le classi (file `.class`, formato bytecode)
 - I file `.class` possono essere trasferiti da librerie, file system locali o nodi remoti in rete
 - Il caricamento avviene dinamicamente quando
 - Viene eseguito il comando `new`, oppure
 - Un oggetto viene trasferito per mezzo di RMI (se il file `.class` non è presente localmente)

Ricostruzione struttura dell'oggetto – Java Runtime Environment



Fonte: <https://docs.oracle.com/cd/E19455-01/806-3461/6jck06gqb/index.html>

- Il file `.class` include:
 - **Struttura e metadati** della classe: nome della classe, da quale **classe** eredita e quali **interfacce** implementa
 - Tabella dei **simboli**: catalogo di tutti i nomi di metodi, variabili e costanti
 - Bytecode dei **metodi**: istruzioni a basso livello relative a ogni metodo
 - Definizione dei **campi**: elenco di tutte le **variabili dichiarate** nella classe (con relativi tipi e modificatori di accesso)

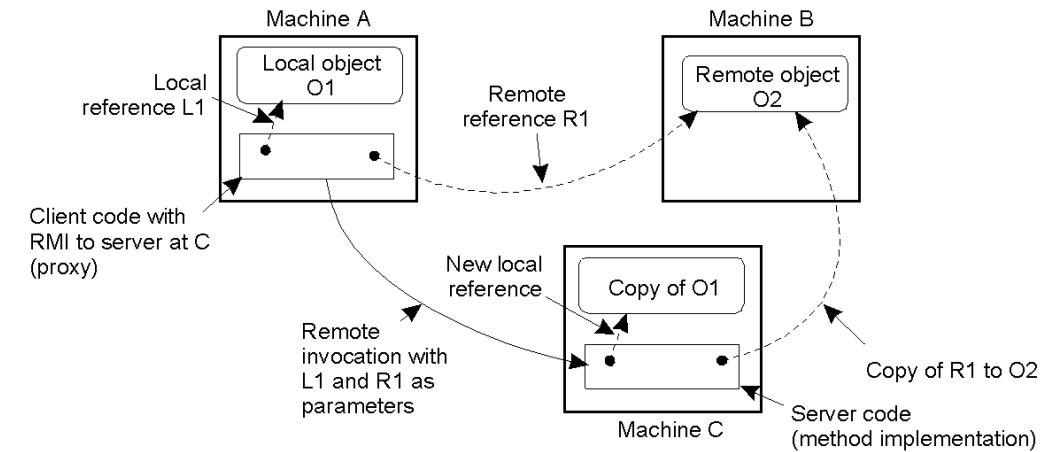
Passaggio dei parametri

■ Passaggio per valore

- Tipi primitivi (`int`, `long`, `boolean`, `float`, ecc.)
- Oggetti non remoti
 - Devono implementare l'interfaccia `java.io.Serializable`

■ Passaggio per riferimento

- Oggetti remoti
 - La classe `java.rmi.server.UnicastRemoteObject` serve allo scopo
 - Implementa le interfacce `java.io.Serializable` e `java.rmi.Remote`



Passaggio dei parametri – Interfacce `Serializable` e `Remote`

- `java.io.Serializable`: specifica che l'oggetto è un «contenitore di dati» e che può essere smontato in una sequenza di byte, spedito in rete e rimontato
 - Passaggio per valore
 - Viene ricevuta una copia esatta dello **stato** (variabili) dell'oggetto originario
- `java.rmi.Remote`: specifica che l'oggetto è remoto e i suoi metodi possono essere invocati da programmi eseguiti su altre macchine
 - Se un oggetto che implementa `Remote` viene passato come parametro in una chiamata remota, non viene inviato l'oggetto vero e proprio ma il suo **stub**
 - Passaggio per riferimento
 - Lo stub è un oggetto includente il **riferimento**, ovvero:
 - Indirizzo IP del server
 - Porta TCP del processo
 - Identificativo univoco dell'oggetto
- **Nota:** la classe `UnicastRemoteObject` implementa `Serializable` per permettere la serializzazione dello stub !

Passaggio dello stato – Serializzazione dell'oggetto

- La serializzazione rappresenta lo stato di un oggetto come **stream di byte**
- È essenziale per poter memorizzare e ricostruire lo **stato** degli oggetti
 - Per definire oggetti persistenti
 - Per trasferire oggetti via rete
- Per mezzo della serializzazione devono essere passate solo le informazioni essenziali sullo stato
 - La descrizione della classe (struttura) è caricata a parte (file `.class`)
- La serializzazione usa il metodo `writeObject(Object obj)` della classe `ObjectOutputStream`
 - Attraversa ricorsivamente tutti i riferimenti ad altri oggetti contenuti in `obj` per costruire una rappresentazione completa del **grafo degli oggetti**
 - Vengono serializzati tutti gli oggetti del grafo (devono implementare `Serializable`)
- Il processo inverso (deserializzazione) usa il metodo `readObject()`

Serializzazione degli oggetti – Esempio (oggetto persistente)

```
1. // serializzazione di stringa e data odierna su un file
2. FileOutputStream f = new FileOutputStream("tmp"); // creazione canale di comunicazione per
   scrittura su file
3. ObjectOutputStream s1 = new ObjectOutputStream(f); // creazione oggetto per serializz.
4. s1.writeObject("Today"); // serializzazione stringa
5. s1.writeObject(new Date()); // serializzazione oggetto data
6. s1.flush(); // passaggio dei dati serializzati da buffer a file (quando buffer non pieno)

7. // deserializzazione di stringa e data da file
8. FileInputStream in = new FileInputStream("tmp"); //creazione canale di comunicazione per
   lettura da file
9. ObjectInputStream s2 = new ObjectInputStream(in); // creazione oggetto per deserializzazione
10. String today = (String)s2.readObject(); // deserializzazione stringa
11. Date date = (Date)s2.readObject(); // deserializzazione oggetto data
```

- **Nota:** il casting nelle righe 10 e 11 è necessario perché `readObject` ritorna un oggetto generico di tipo `Object`
- **Importante!** Il file `tmp` potrebbe senza problemi essere trasferito via rete su una macchina remota per la deserializzazione!

Serializzazione degli oggetti

- I tipi primitivi (`int`, `long`, `boolean`, `float`, ecc.) sono serializzabili in modo nativo
- È possibile creare classi serializzabili implementando l'interfaccia `Serializable`
- E' possibile ridefinire i metodi

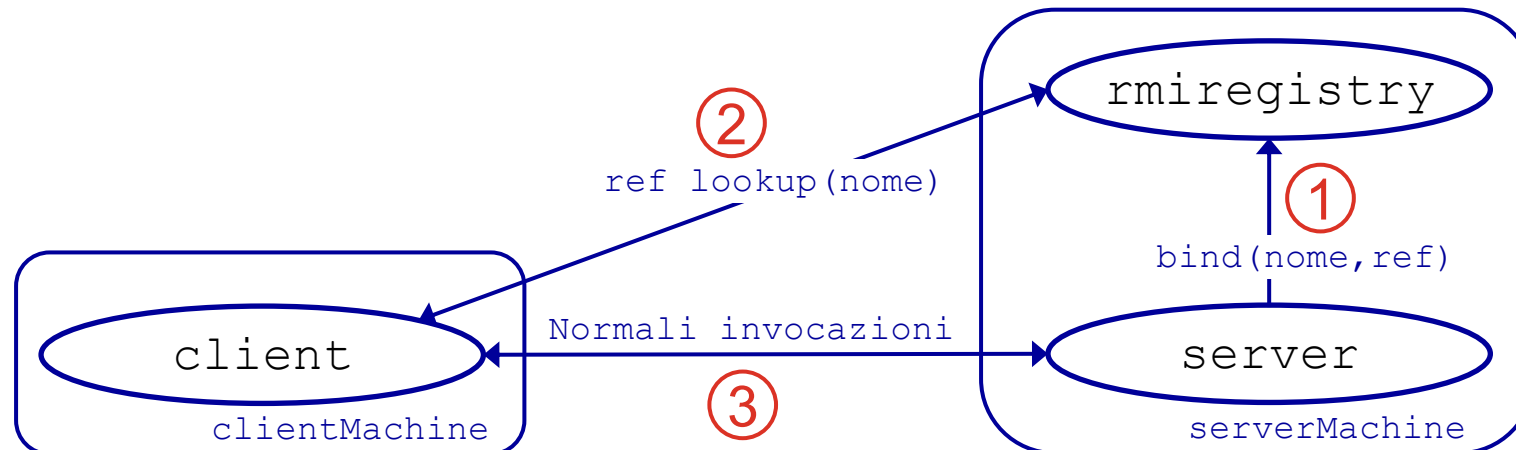
```
private void writeObject(java.io.ObjectOutputStream out)
                        throws IOException;
```

```
private void readObject(java.io.ObjectInputStream in)
                        throws IOException, ClassNotFoundException;
```

- La ridefinizione di tali metodi permette di controllare manualmente la serializzazione
 - Es: crittare una password prima della serializzazione

Passaggio dei riferimenti

- RMI definisce un componente chiamato **RMI registry** (`rmiregistry`)
 - Si ha un `rmiregistry` su ogni macchina che ospita oggetti remoti
 - Associa univocamente un nome logico assegnato all'oggetto al suo riferimento
 - RMI attiva un servizio di ascolto per quel servizio
 - Convenzionalmente alla porta 1099 (TCP)
- Viene utilizzata la classe `java.rmi.Naming` per accedere alle funzionalità dell'RMI registry
 - Metodi fondamentali: `bind` e `lookup`



La classe `java.rmi.Naming`

■ Parametri:

- `name`: nome logico dell'oggetto in formato URL
 - `[[rmi://]<host_name>:<name_service_port>/]<service_name>`
 - Se non specificati, l'`host_name` di default è `localhost` e la `name_service_port` è `1099`
- `obj`: il riferimento all'oggetto remoto (solitamente un oggetto **stub**)

■ Metodi:

- `public static void bind(String name, Remote obj)`
`throws AlreadyBoundException, MalformedURLException, RemoteException`
 - Collega il nome specificato all'oggetto remoto
- `public static void rebind(String name, Remote obj)`
`throws RemoteException, MalformedURLException`
 - Collega (bind) il nome specificato all'oggetto remoto, cancellando i collegamenti esistenti
- `public static Remote lookup(String name)`
`throws NotBoundException, MalformedURLException, RemoteException`
 - Restituisce un riferimento (stub) all'oggetto associato al nome specificato

La classe `java.rmi.Naming`

■ Metodi (continua):

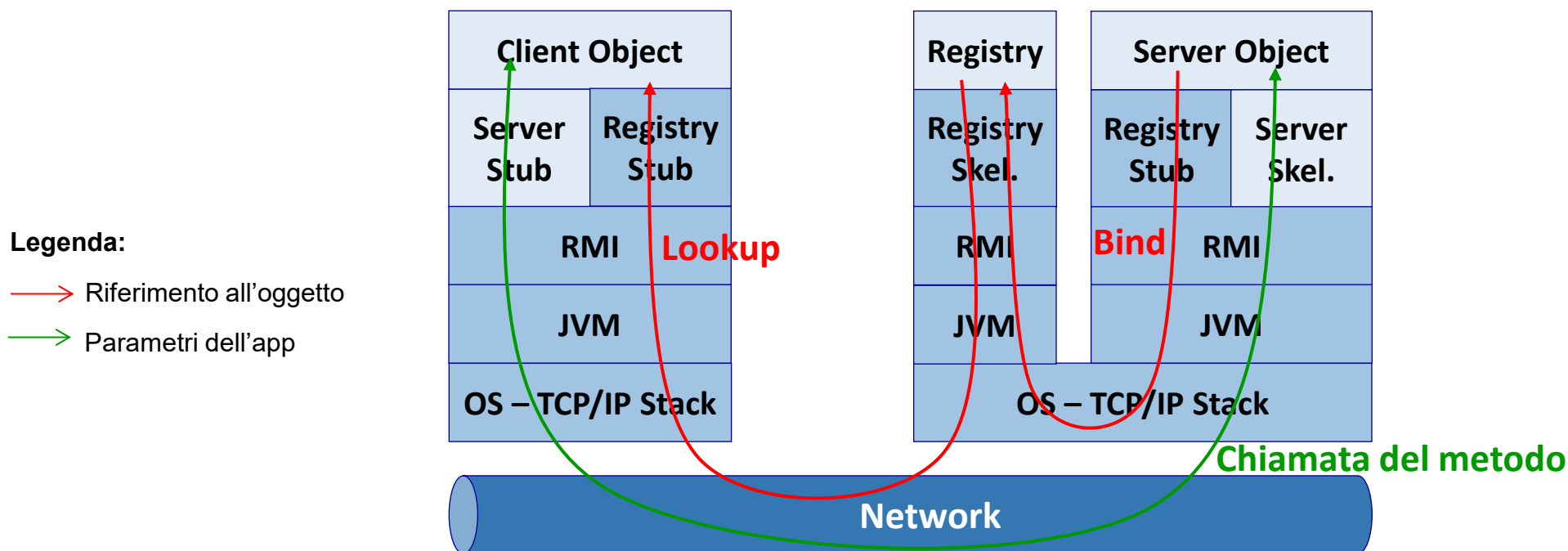
- `public static String[] list(String name)`
`throws RemoteException, MalformedURLException`
 - Restituisce i nomi (in formato URL) degli oggetti del registry
- `public static void unbind(String name)`
`throws RemoteException, NotBoundException, MalformedURLException`
 - Distrugge il collegamento (bind) al nome specificato

■ Eccezioni:

- `AlreadyBoundException`: il nome è già collegato a un altro oggetto
- `NotBoundException`: il nome non è collegato ad alcun oggetto
- `RemoteException`: il registry non può essere contattato
- `MalformedURLException`: il nome non è un URL formattato in modo appropriato

Passaggio dei riferimenti – Workflow in dettaglio

- Il server pubblica il riferimento e il nome dell'oggetto remoto nel registry invocando il metodo `bind`
- Il client ottiene il Server Stub e il riferimento all'oggetto remoto invocando il metodo `lookup`
- Il client invoca il metodo sullo stub, il quale inoltra al server il riferimento dell'oggetto remoto e l'identificativo del metodo da eseguire, più i parametri del metodo
- **Nota:** il registry ha stub e skeleton → E' un oggetto remoto con cui bisogna comunicare!



Problematiche fondamentali dei sistemi distribuiti e Java RMI

- In che modo le problematiche fondamentali dei sistemi distribuiti sono affrontate quando Java RMI è adottato per l'invocazione di metodi di oggetti remoti?

1. Identificazione della controparte (**naming**)

- Sono necessari l'hostname e il nome dell'oggetto

2. Accedere alla controparte (**access point**)

- Accesso per mezzo del RMI registry
 - Traslazione della coppia `<hostname, nome oggetto>` nel riferimento all'oggetto

3. Comunicazione 1 (**protocol**)

- Serializzazione degli oggetti locali o riferimento a oggetti remoti

4. Comunicazione 2 (**syntax and semantics**)

- Tipi di Java (classi e metodi)

Problematiche fondamentali dei sistemi distribuiti e Java RMI

- Adottando Java RMI per l'implementazione di un sistema distribuito, qual è il **livello di trasparenza** per il programmatore?
 - **Alto**, in quanto rende gli oggetti remoti simili a quelli locali
- Il programmatore deve comunque gestire la natura parziale e instabile dei sistemi distribuiti tramite l'uso esplicito delle eccezioni
 - Eventuali problemi di rete devono essere previsti e gestiti esplicitamente

Creare un oggetto remoto – Dettagli

- Come già detto, Java definisce un'interfaccia specifica per implementare oggetti remoti:
`interface java.rmi.Remote`

- Per creare una classe remota bisogna:

1. Definire l'interfaccia della classe remota (estensione di `Remote`)
2. Implementare la nuova interfaccia
3. Implementare un server che crei l'oggetto e lo registri al registry

- Esempio:

1. `MyRemoteObject myObj = new MyRemoteObject(); // creazione dell'oggetto remoto`
2. `Registry localRegistry = LocateRegistry.getRegistry(); // creazione stub registro`
3. `localRegistry.bind("publicName", myObj); // associazione nome-riferimento nel registro`

Nota: non avendo passato parametri a `getRegistry()`, viene assunto come indirizzo `localhost` e come porta la 1099

Un semplice esempio – Applicazione echo

- Realizzazione di una semplice applicazione: `echo`
- Riceve una sequenza di byte dal client e la restituisce al client
- Esempio di funzionamento: un client contatta a turno 5 oggetti remoti, istanze di una classe (`EchoImpl`) che fa la eco dei messaggi ricevuti

1. Definire l'interfaccia della classe remota (es. file `Echo.java`)

- Si estende l'interfaccia `Remote`

```
1. public interface Echo extends java.rmi.Remote {  
2.     public String call(String message)  
3.         throws java.rmi.RemoteException;  
4. }
```

- Il metodo `call` riceve il messaggio che verrà rimbalzato
- **Nota:** ogni metodo dell'interfaccia deve lanciare `java.rmi.RemoteException`
 - I dati viaggiano in rete!

Un semplice esempio – Applicazione echo

2. Implementare la nuova interfaccia (es. file `EchoImpl.java`)

- Si realizza la nuova classe remota (`EchoImpl`)
 - Si estende la classe `UnicastRemoteObject`
- Gerarchia delle classi:

```
class java.rmi.server.RemoteObject
    (implements java.rmi.Remote, java.io.Serializable)
class java.rmi.server.RemoteServer
    class java.rmi.activation.Activatable
        class java.rmi.server.UnicastRemoteObject
```

3. Implementare un server che crei l'oggetto remoto e lo registri al registry (es. file `EchoImpl.java`)

- Creare un oggetto che realizza il servizio desiderato (oggetto del tipo `EchoImpl`)
- Registrare l'oggetto presso l'RMI registry (sullo stesso host) con un nome pubblico

Implementazione lato server – EchoImpl (1)

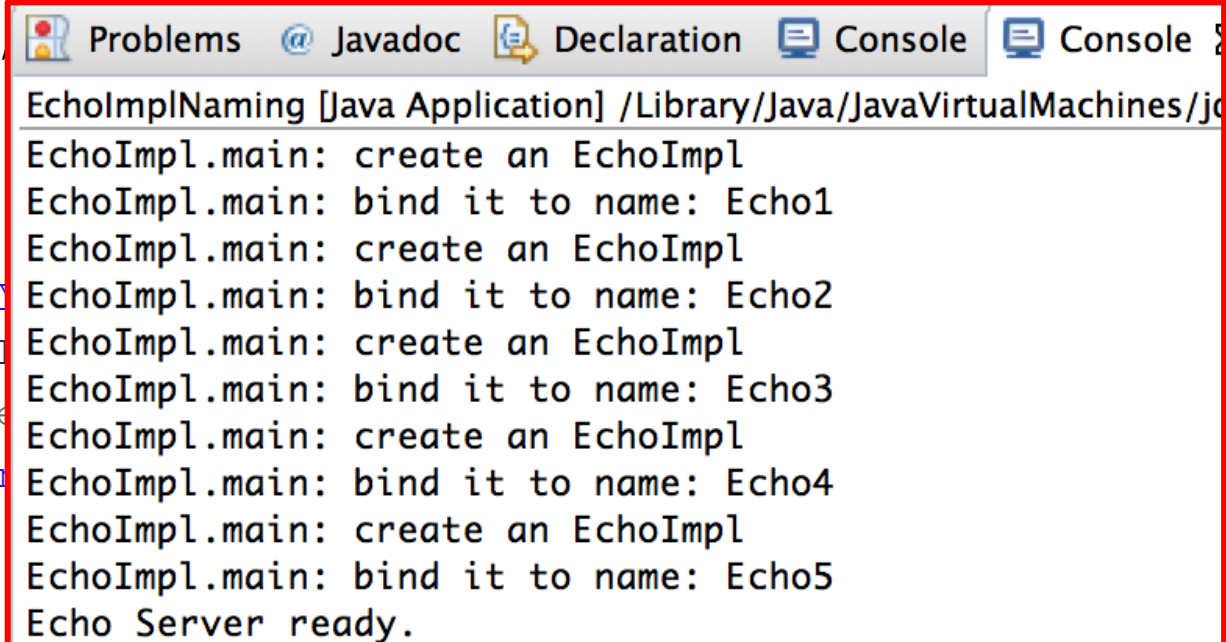
```
1. import java.rmi.server.UnicastRemoteObject;
2. import java.rmi.Naming;
3. import java.rmi.RemoteException;
4. import java.rmi.MalformedURLException;
5. import java.rmi.registry.LocateRegistry;
6. // estensione della classe UnicastRemoteObject che implementa l'interfaccia Echo
7. public class EchoImpl extends UnicastRemoteObject implements Echo
8. {
9.     private String name;
10.    // il costruttore è obbligatorio se estende UnicastRemoteObject perché deve lanciare
11.    // obbligatoriamente l'eccezione RemoteException
12.    public EchoImpl(String s) throws RemoteException {
13.        super(); // invoca il costruttore di UnicastRemoteObject (oggetto esposto in rete)
14.        name = s; // nome dell'oggetto che viene creato
15.    }
16.    // implementazione del metodo call (interfaccia Echo)
17.    // restituisce un messaggio con nome per poter discriminare il server che risponde
18.    public String call(String message) throws RemoteException {
19.        return name + " received: " + message;
20.    }
```

Implementazione lato server – EchoImpl (2)

```
21. public static void main(String args[]) {
22.     try {
23.         LocateRegistry.createRegistry(1099); // crea il registro, in ascolto su porta 1099
24.         for (int i = 1; i <= 5; i++) { // cicla sui cinque oggetti remoti
25.             System.out.println("EchoImpl.main: create an EchoImpl");
26.             String name = "Echo" + i; // nome dell'oggetto
27.             EchoImpl echo = new EchoImpl(name); // istanzia l'oggetto remoto EchoImpl
28.             System.out.println("EchoImpl.main: bind it to name: " + name);
29.             Naming.rebind(name, echo); // crea associazione nome-oggetto remoto nel registro
30.         }
31.         System.out.println("Echo Server ready.");
32.     } catch (RemoteException | MalformedURLException e) {
33.         // gestione eccezioni (errori di rete (RMI) o URL del registro errato)
34.         System.out.println("EchoImpl.main: an exception occurred: " + e.getMessage());
35.         e.printStackTrace();
36.     }
37. }
38. }
```

Implementazione lato server – EchoImpl (2)

```
21. public static void main(String args[]) {
22.     try {
23.         LocateRegistry.createRegistry(1099); // crea il registro, in ascolto su porta 1099
24.         for (int i = 1; i <= 5; i++) { // cicla sui cinque oggetti remoti
25.             System.out.println("EchoImpl.main: create an EchoImpl");
26.             String name = "Echo" + i; // nome dell'oggetto
27.             EchoImpl echo = new EchoImpl(name);
28.             System.out.println("EchoImpl.main:");
29.             Naming.rebind(name, echo); // crea
30.         }
31.         System.out.println("Echo Server ready");
32.     } catch (RemoteException | MalformedURLException |
33.             // gestione eccezioni (errori di rete)
34.             System.out.println("EchoImpl.main: an
35.             e.printStackTrace();
36.     }
37. }
38. }
```



Problems Javadoc Declaration Console Console

EchoImplNaming [Java Application] /Library/Java/JavaVirtualMachines/jc

EchoImpl.main: create an EchoImpl
EchoImpl.main: bind it to name: Echo1
EchoImpl.main: create an EchoImpl
EchoImpl.main: bind it to name: Echo2
EchoImpl.main: create an EchoImpl
EchoImpl.main: bind it to name: Echo3
EchoImpl.main: create an EchoImpl
EchoImpl.main: bind it to name: Echo4
EchoImpl.main: create an EchoImpl
EchoImpl.main: bind it to name: Echo5
Echo Server ready.

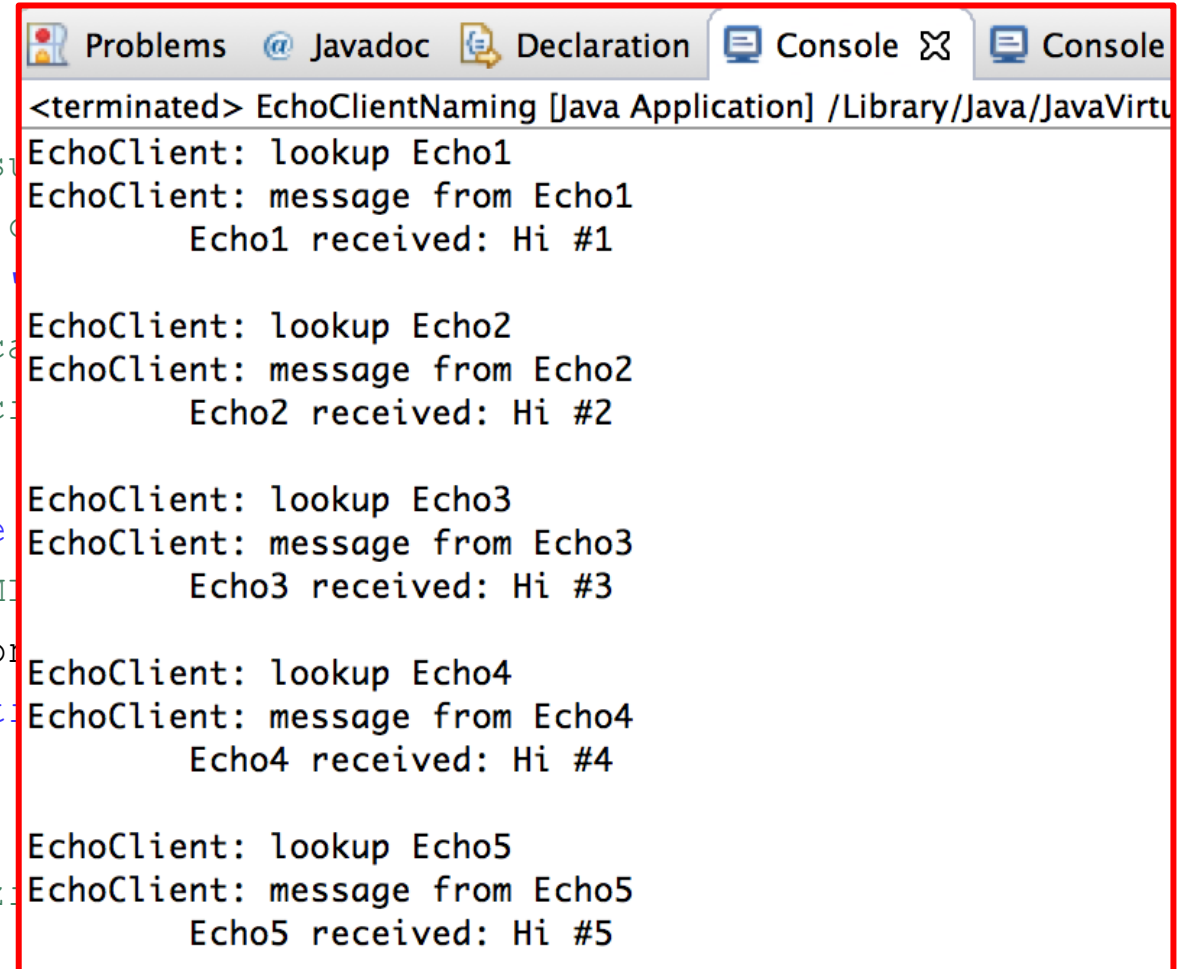
Domanda: quanto il main termina, cosa succede agli oggetti remoti creati?

Implementazione lato client – EchoClient

```
1.  import java.rmi.Naming; import java.rmi.NotBoundException;
2.  import java.rmi.RemoteException; import java.net.MalformedURLException;
3.  public class EchoClient {
4.      public static void main(String args[]) {
5.          try {
6.              for (int i = 1; i <= 5; i++) { // cicla su tutti gli oggetti remoti
7.                  String name = "Echo" + i; // nome dell'oggetto da contattare
8.                  System.out.println("EchoClient: lookup " + name);
9.                  Echo echo = (Echo) Naming.lookup("//localhost:1099/" + name); // creazione dello stub
10.                 // nota: necessario casting a interfaccia (lookup restituisce oggetto tipo Remote)
11.                 String message = echo.call("Hi #" + i); // invocazione metodo remoto
12.                 System.out.println("EchoClient: message from " + name + "\t" + message + "\n");
21.            } // gestione eccezioni (errori di rete (RMI), oggetto non esistente, URL del registro errato)
1.        } catch (RemoteException | NotBoundException | MalformedURLException e) {
2.            System.out.println("EchoClient: an exception occurred: " + e.getMessage());
3.            e.printStackTrace();
4.        }
5.        System.exit(0); // interruzione dell'esecuzione della JVM (tutto ok)
6.    }
7. }
```

Implementazione lato client – EchoClient

```
1. import java.rmi.Naming; import java.rmi.NotBoundException;
2. import java.rmi.RemoteException; import java.net.MalformedURLException;
3. public class EchoClient {
4.     public static void main(String args[]) {
5.         try {
6.             for (int i = 1; i <= 5; i++) { // cicla su 5
7.                 String name = "Echo" + i; // nome dell'echo
8.                 System.out.println("EchoClient: lookup " + name);
9.                 Echo echo = (Echo) Naming.lookup("//localhost:1099/");
10.                // nota: necessario casting a interfaccia Echo
11.                String message = echo.call("Hi #" + i);
12.                System.out.println("EchoClient: message received: " + message);
21.            } // gestione eccezioni (errori di rete (RMI))
1.        } catch (RemoteException | NotBoundException e) {
2.            System.out.println("EchoClient: an exception occurred");
3.            e.printStackTrace();
4.        }
5.        System.exit(0); // interruzione dell'esecuzione
6.    }
7. }
```



Problems Javadoc Declaration Console Console

<terminated> EchoClientNaming [Java Application] /Library/Java/JavaVirtualMachine

EchoClient: lookup Echo1
EchoClient: message from Echo1
Echo1 received: Hi #1

EchoClient: lookup Echo2
EchoClient: message from Echo2
Echo2 received: Hi #2

EchoClient: lookup Echo3
EchoClient: message from Echo3
Echo3 received: Hi #3

EchoClient: lookup Echo4
EchoClient: message from Echo4
Echo4 received: Hi #4

EchoClient: lookup Echo5
EchoClient: message from Echo5
Echo5 received: Hi #5

Compilazione ed esecuzione dell'esempio

■ In Windows o Linux:

- Inserire le classi `Echo`, `EchoImpl` e `EchoClient` in una stessa directory `echo` e compilare
 - La riga 23 di `EchoImpl` va rimossa (`LocateRegistry.createRegistry(1099);`)
 - A seguito della compilazione saranno presenti i rispettivi file `.class`
- Attivare tre terminali nella directory `echo`
- Terminale 1, attivare il registry:
> `start rmiregistry` in Windows (> `rmiregistry` in Linux)
- Terminale 2, attivare il server che crea gli oggetti remoti:
> `java EchoImpl`
- Terminale 3, attivare il client:
> `java EchoClient`

■ In Eclipse:

- La riga 23 di `EchoImpl` deve essere presente (`LocateRegistry.createRegistry(1099);`)
- Eseguire `EchoImpl` e `EchoClient` come applicazioni java

Riferimenti bibliografici

- M. Van Steen, A. S. Tanenbaum, Distributed Systems, 4th Edition
 - Sezione 4.2 («Remote Procedure Call»)
- Sito web di gRPC: <https://grpc.io/>
- Java Tutorial – Trail RMI: <https://docs.oracle.com/javase/tutorial/rmi/>
 - L'applicazione `echo` è originale ma prende spunto dal tutorial
- Documentazione Java SE API (es. `java.rmi`, `java.rmi.registry`, `java.rmi.server`, `ClassLoader`, ...): <https://docs.oracle.com/javase/8/docs/api/>